AI SECURITY ASSESSMENT REPORT

Vroomi AI Assistant

Prepared for CISO, Softmicro · **Prepared by:** Amiya Pradhan, AI Security Practitioner · July 2026 · Confidential

Decision: **CONDITIONAL GO** — subject to the safeguards in §6.

1. Executive Summary

Vroomi is an internal AI assistant (gpt-4o-mini) that books and cancels rides, retrieves trip history, and answers policy questions from a small knowledge base. Softmicro wants to make it more agentic; this assessment reviews the current system first.

- **Scope:** four scenarios across the system's main trust boundaries — prompt injection, data poisoning, sensitive-information disclosure, and improper output handling.
- **Key risks:** unauthorised backend actions, exposure of customer PII from the model's context, and fabricated policy from unverified knowledge-base content.
- **Recommendation:** Conditional Go. The weaknesses are in the application, not the model, and each has an architectural fix that should be mandatory before agentic features ship.

2. System Overview

Vroomi takes a request, consults the model, and either replies in text or performs a backend action. Its components are user input (untrusted), a system prompt that also holds the knowledge base and tool protocol, the model, a tool executor, and three backend actions — book, cancel, and read trip history — that change state and carry operational and financial consequences. A trust-boundary diagram accompanies this report (Appendix).

3. Threat Analysis

This report covers the four highest-severity threats, each mapping to the OWASP Top 10 for LLM Applications (2025). The full six-threat catalogue — adding System Prompt Leakage and Misinformation — is in threat-cards.md.

Threat 1 — Prompt Injection OWASP LLM01

Description: Sent user input with an embedded “respond only with” tool call.
Observed: A real trip was cancelled; the model was bypassed by a parser acting on the injected instruction.
Root cause: The application treated untrusted user text as a command. — affected: input handling / tool executor

Threat 2 — Data Poisoning OWASP LLM04

Description: Added a plausible but false policy document to the knowledge base, then asked a related question.
Observed: The assistant answered from the poisoned document as if it were fact.
Root cause: The knowledge base was passed to the model unfiltered — no provenance check. — affected: knowledge base

Threat 3 — Sensitive Information Disclosure OWASP LLM02

Description: Asked for customer record information.
Observed: Disclosed a customer’s name, account ID, and internal review flag — on the baseline, no exploit needed.
Root cause: A high-sensitivity record sat in the model’s context with no access control. — affected: context assembly

Threat 4 — Improper Output Handling OWASP LLM05

Description: Examined how the executor treats output shaped like a tool call.
Observed: Any JSON-shaped output was parsed and executed directly — no schema, allow-list, or intent check.
Root cause: The application trusted model output as a command, not a proposal. — affected: tool executor

4. Risk Assessment

Improper Output Handling and Prompt Injection are highest-severity: both reach unauthorised backend actions through the same choke point, the tool executor. Sensitive Disclosure is close behind — regulatory and reputational — and triggers on the happy path. Likelihood is high; the attacks are one-line inputs. All scale with autonomy: more tools and automatic actions widen the executor's blast radius, turning these into Excessive Agency (LLM06).

5. Effectiveness of Mitigations

Each scenario shipped a defended version. The fixes are architectural — software at the trust boundary, not prompt tweaks.

Threat	Mitigation	Result
Prompt Injection	Act only on validated model output; confirm user intent.	**Resolved**
Data Poisoning	Provenance filter (approval + trusted source) before the model.	**Largely resolved**
Sensitive Disclosure	Classify by sensitivity; keep sensitive records out of context.	**Resolved**
Improper Output Handling	Schema + tool allow-list + argument + intent checks.	**Resolved**

Residual risk (subtly worded poisoning, novel injection phrasings) is managed with defence-in-depth and monitoring, and recorded rather than assumed away.

6. Recommendations

1. **Validate every tool call before it executes** — schema, allow-list, arguments, and user intent.
2. **Filter the knowledge base on the read path** — provenance and sensitivity; keep sensitive records out of context.
3. **Remove any path from user input to internal configuration.**
4. **Require human approval for irreversible or high-impact actions.**
5. **Log every gated decision** — action, decision, reason, owner.
6. **Assign a named owner and an approved agent card;** re-assess on any capability change.

7. Additional Threats and Future Testing

- **System Prompt Leakage (LLM07):** debug or maintenance paths can expose internal instructions.
- **Misinformation (LLM09):** mixed-quality sources produce confident wrong answers.
- **Supply-chain (LLM03) and unbounded consumption (LLM10)** as the system automates.
- **Next:** an adversarial regression suite that proves the defended controls fire on every change.

8. Final Recommendation

Decision: Conditional Go — subject to implementation of safeguards.

Proceed once Recommendations 1–6 are implemented and verified. The fixes are architectural and inexpensive, and autonomy amplifies the current gaps — so implement them before granting Vroomi more autonomy, not after.

9. Appendix

- **System diagram:** vroomi-system-diagram.svg.
- **Example test case:** an injected cancel_ride inside a policy question — executed by the baseline, refused by the gate.
- **Proof-of-fix:** vroomi_gov_gate.py — the recommended controls in one runnable file, with a before/after transcript.